COMP 530 Assignment 8: Putting It All Together

1. The Task

Your task, very simply, is to get the database to run! All of the pieces we've built this semester will fit together into a single, final product. Basically, when your database is given an SQL query, the system will compile it, optimize it using the A7 optimizer, and then execute the relational algebra, print the results, and then clean up any garbage files that you have created.

Note that in `A8.zip`, I've included my solution to A7. That this code maintains a `MyDB_TableReaderWriter` object for each of the tables currently defined in the system (it also maintains a `MyDB_BPlusTreeReaderWriter` object for each B+-Tree file), and these are passed into the optimized tree of LogicalOp objects for execution. The tree will use these ReaderWriter objects to access the data stored in the database.

The database also has some functionality for loading those tables from text (for example, to create the `supplier` table you'd type "`LOAD supplier FROM supplier.tbl`"). Also note that it is possible to create a table using a B+-Tree, by simply adding "`AS BPLUSTREE ON some_attribute`" at the end of the `CREATE TABLE` statement. Note that I did not include my own type checking code in `A8.zip`, because all of the queries in the test suite are semantically correct.

2 Creating the TPC-H Data

For A8, we'll be using the TPC-H benchmark database. You can either create this database yourself (see `http://www.tpc.org/tpch/`) or you can download a version that I've posted onto Dropbox, at:

https://www.dropbox.com/scl/fo/6z15bjw0dr0n7sxnxvj15/h?rlkey=uvc974dtnalru71bxkubyimix&st=x5vi201v&dl=0

Note that we are using the default "scale factor one" database, which is about one GB in size.

What you should do is to first download the TPC-H files, and then create the tables. Next you load all of the files by executing a "`LOAD xxx FROM xxx.tbl`" command for each of the tables. Exit the database type typing "`exit;`". Now everything is saved and when you fire up the database again you don't need to load or create the tables again.

3 The Path to Finishing the Assignment

The first thing you'll need to do is to look over the code that I have provided. As you look over the code, keep in mind that your goal is to fill out just two methods: `LogicalTableScan :: execute` and `LogicalJoin :: execute`. In the last

assignment, you wrote code to build up trees of `LogicalTableScan` and `LogicalJoin` objects; here you are adding code that will traverse such a tree and execute the query. Once this has been done, you will be able to run queries.

Both methods you need to implement are tasked with setting up the call to the appropriate relational implementation to execute the logical operation. In the case of the `LogicalJoin`, you will first execute the left hand side, then execute the right hand side, then execute the join (scan join or sort merge; this involves calling an appropriate `RelOp` operation). In the case of the `LogicalTableScan`, you will execute a table scan, or you can use a B+ tree if appropriate.

As you work on the assignment, here are some things to keep in mind:

1) Make sure to clean up tables as you are done with them so you don't leave a bunch of temporary files around all over the place! There is a `killTable` method on the buffer manager that can do this.

2) If you want to, you can add functionality to detect when a B+-Tree can be used to help answer a query by using it to implement a fast table scan (call this as part of the `LogicalTableScan`'s `execute` method) but this is not necessary.

3) Another optimization is to avoid running a `LogicalTableScan` that is to the left or to the right of a `LogicalJoin`. You can simply add the selection predicate in the input to the join and skip the `LogicalTableScan`. This has the advantage of making it so that you don't write out an additional temporary table. But again, this is not necessary.

4) The "worst" queries can take a few minutes to run. If you want to know that they are making progress during debugging, you might want to add some print statements that occasionally displays a number of records processed so far to the screen during the join. Please take that debugging information out as you prepare your output document, however!

5) Note that in the `LogicalJoin`, you will need to add some logic to choose whether to run a scan join or a sort-merge join. Choose the latter if the smaller of the two tables takes up more than half of the pages available to the buffer manager.

6) To compile and run your assignment, run scans and choose option 11 to make the SQL compiler. Executing the resulting program will execute the database.

4 Testing, Grading, and Turnin

Your grade will be based upon a turnin doc that you need to submit separately from your code. This needs to be in PDF (while we will grade your PDF doc, we'll also be running a few of the test queries using your database, just to make sure that everything works correctly).

Included with the assignment is a suite of twelve test queries. When you are ready to

prepare your turnin doc, you should fire up your program and run those queries (in order) in single session. Copy and paste the output of your system into your doc. Skip any queries that you can't run.

For turnin, you need to include the PDF file that includes all of the results you get from running those 12 queries . Make sure it has both team member names. This should be turned in along with all of your source code as a zip archive. Please turn in two different files: one PDF, and one zip.

And also: if you work with a partner, **only turn in one submission**. Otherwise, we'll possibly grade your submission twice, and end up getting quite annoyed.

5 A Note on the Due Date

Note that the assignment is due on the last day of the semester. This is a busy time for everyone, but unfortunately, there are no extensions on this assignment as we can't have it due past the end of the semester.

6 A Note on Space

Many people will want to run this on Clear. If this is the case, you will likely run out of space (that is, running these queries will cause you to exceed your quota, because the database tables are large, and some of the joins will produce a lot of data). I will send in a request to have your quota increased for the next couple of weeks so that you can work on the assignment.